

Enhancing Multivariate Time Series Forecasting: A Novel Approach with Mallows Model Averaging and Graph Neural Networks*

ZHANG Haili · WANG Jiawei · LIU Zhaobo · DONG Hailing

DOI: 10.1007/s11424-024-4044-9

Received: 31 January 2024 / Revised: 25 April 2024

©The Editorial Office of JSSC & Springer-Verlag GmbH Germany 2025

Abstract Multivariate time series forecasting holds substantial practical significance, facilitates precise predictions, and informs decision-making. The complexity of nonlinear relationships and the presence of higher-order features in multivariate time series data have sparked a burgeoning interest in leveraging deep learning approaches for such forecasting tasks. Existing methods often use pre-scaled neural networks, whose reliability and generalization can pose a challenge. In this study, the authors propose an instance-wise graph-based Mallows model averaging (IGMMA) framework for multivariate time series prediction. The framework incorporates a model averaging module into the network, where extracted features are utilized as inputs for candidate linear models. These linear models are combined with weights to create a new linear layer, forming a novel graph neural network model. Moreover, the network loss function is modified based on the Mallows criterion, where penalties are imposed on the parameters and the weights separately. The authors use the proposed method to predict multicommodity futures prices, and the empirical results show that IGMMA has superior predictive accuracy even when small neural networks are used. This indicates that the model averaging module significantly reduces the parameters required for deep learning training, which enables the training of multiple small models as an alternative to training a large model.

Keywords Graph neural networks, model averaging, multivariate time series.

ZHANG Haili

School of Undergraduate Education, Shenzhen Polytechnic University, Shenzhen 518055, China.

Email: zhanghl@szpu.edu.cn.

WANG Jiawei · LIU Zhaobo (Corresponding author)

Institute for Advanced Study, Shenzhen University, Shenzhen 518060, China.

Email: iwnlgre@outlook.com; liuzhaobo@szu.edu.cn.

DONG Hailing

Science of Mathematics College, Shenzhen University, Shenzhen 518060, China. Email: hailing@szu.edu.cn.

*This research was supported by the Research Foundation of Shenzhen Polytechnic University under Grant No. 6023312034K, the Post-doctoral Later-Stage Foundation Project of Shenzhen Polytechnic University under Grant No. 6023271021K, the National Science Foundation of Guangdong Province of China under Grant No. 2024A1515011542, the Guangdong Provincial Ordinary Universities Youth Innovative Talent Project (Natural Science) under Grant No. 2023KQNCX064, the National Natural Science Foundation of China under Grant No. 12371448, and the National Key R&D Program of China under Grant No. 2023YFE0126800.

◇ This paper was recommended for publication by Editor LIAO Jun.

1 Introduction

Multivariate time series prediction has a wide range of applications in various domains, including traffic road load^[1], power consumption^[2], and financial asset price prediction^[3]. The dynamic nature of multivariate time series data, with its complex dependencies and changing patterns, presents a significant challenge for traditional forecasting models. Deep learning has become a solution to this challenge because it can capture intricate and complicated patterns and relationships in the data^[4].

In recent years, deep learning techniques have been used extensively for multivariate time series forecasting because of advances in computing power and data processing technology^[5–7]. These approaches involve the construction of neural network models that automatically learn data patterns and make predictions about multivariate time series. For example, in the case of futures prices, Wang and Gao^[5] used the Dalian Commodity Exchange dataset to build a long- and short-term memory (LSTM) neural network model to predict high and low soybean futures prices. Similarly, Sun, et al.^[6] employed deep LSTM as their primary architecture and expanded market trading data to predict trends in the futures market. Ji, et al.^[7] used the hybrid ARIMA-CNN-LSTM model to predict futures prices based on carbon trading.

However, the neural network models mentioned above often require a preset model scale, which is essentially equivalent to model selection. Model selection often leads to unstable estimates, where even slight perturbations in the data can result in the selection of a completely different model^[8, 9]. Furthermore, model selection may lose valuable information^[10]. Some model selection methods are intended to select the model that best reflects the true data generation process but may not necessarily provide reliable estimation or predictive performance^[11]. For deep neural networks, selecting a fixed neural network model can result in overfitting and poor generalization performance^[12, 13], thereby limiting the performance of models in practical applications.

Model averaging, which is an alternative to model selection, involves the weighted averaging of estimates^[14] or predictions from different models using specific weights. This approach prevents the selection of poor models for prediction, thereby improving prediction reliability^[15]. By weighting a large number of neural network models of different scales during the training process, this study aims to retain the advantages of each model, reduce overfitting risks, and improve the robustness of predictions and the generalization performance of models on different datasets. The existing model averaging methods are mainly studied in two directions: Bayesian model averaging (BMA) and frequentist model averaging (FMA). Among them, the BMA method has been widely used in deep neural networks^[16–18], but with the increase of data size and number of parameters, the calculation of approximate integration in Bayesian methods is difficult to handle, and specifying a meaningful prior for deep neural networks is another important challenge. Compared with the BMA method, FMA is completely data-dependent and more flexible. The FMA method for deep neural networks in the existing literature can refer to ensemble deep learning^[19]. Since the ensemble learners train independently from each other, such methods are easily parallelized, and the main challenge is the need to introduce

diversity among the ensemble learners. Different from the existing literature, this paper will combine model averaging methods and deep neural networks from a new perspective. We add the model averaging part as a new linear layer for deep neural networks. It is noted that for linear models, the model averaging estimators based on some specific selection criterion for assigning weights are mathematically proven to be able to achieve the asymptotic optimality of predicting risk^[20], so it is possible to obtain the optimal combination prediction of deep neural networks at different scales in the sense of predicting risk by introducing specific criterion to assist in the training of the added linear layers.

Various methods have been proposed regarding the selection criteria for assigning weights in model averaging. In this paper, we focus on the model averaging technique based on the Mallows criterion, as it is the first criterion to be shown to have asymptotic optimality. The Mallows model averaging (MMA) method proposed by Hansen^[21], is a computationally efficient method for selecting the weights of a combined nested linear model by minimizing the Mallows criterion. This method was found to have asymptotic optimality for model averaging estimation under the assumption of a discrete weight space. Later, Wan, et al.^[22] generalized the conclusion in [21] and proved that model averaging estimation based on the Mallows criterion still has asymptotic optimality under continuous weight space and for general combined models. For heteroscedasticity, Liu and Okui^[23] proposed a modified MMA method^[21]. In the setting of distributed data, Zhang, et al.^[24] proposed a corresponding MMA estimator and successfully demonstrated its convergence of weights, asymptotic optimality of predictive risk, and the minimax property of the estimator. For more complex statistical models, such as threshold autoregressive models, varying coefficient partial linear models, and orthogonal kriging models, please refer to the references [25–27]. Qiu, et al.^[28] extended the MMA theory to combine it with traditional machine learning methods and established the asymptotic optimality in the sense of achieving the lowest possible squared error. Overall, the current research on MMA is mainly focused on statistical models or traditional machine learning models; there is a notable lack of research investigating the integration of MMA methods into the deep learning framework. It is worth exploring whether MMA methods can improve the predictive ability of deep neural networks, which have a large parameter space and high nonlinearity.

In this study, we propose an instance-wise graph-based Mallows model averaging (IGMMA) framework that incorporates MMA methods into a deep learning framework for the prediction of multivariate time series. This framework uses deep graph neural networks for feature extraction and applies Mallows criterion to weight models that correspond to different feature numbers. In terms of empirical analysis, we investigate the problem of predicting multicommodity futures price sequences. We conduct ablation experiments using the model averaging module and empirical studies using the modified IGMMA framework with an optimized loss function. The main contributions of this study are as follows:

- 1) Unlike the existing research on model averaging methods in deep learning, we construct a new model averaging module based on features extracted by deep learning models. This innovative approach allows for the integration of information criteria with good theoretical

properties, commonly used in statistical model averaging, into deep learning. As an initial exploration, we utilize the widely adopted and theoretically sound Mallows criterion to develop a novel graph neural network learning framework based on MMA.

- 2) Through empirical analysis, we find that after incorporating the model averaging module, the IGMMA framework can achieve higher prediction accuracy with fewer parameters than several typical deep learning prediction methods. This suggests that the inclusion of a model averaging module may potentially alleviate the difficulties associated with training deep neural networks, thereby providing broader application prospects in other deep learning problems.

This paper is structured as follows. In Section 2, we present the IGMMA framework and algorithm. Subsequently, an empirical analysis is conducted to evaluate the prediction performance of the algorithm and to optimize the hyperparameters of the algorithm in Section 3. Finally, Section 4 provides a summary of the work and outlines the prospects of this study.

2 IGMMA Framework

In this section, we introduce our proposed method for multivariate time series prediction. A multivariate time series prediction is a statistical technique that considers the interrelationships and influences between different variables. Each variable value at time t is influenced not only by its historical values but also by the historical values of other variables. Due to the similarity of external environments or the periodicities in time series, one variable may follow a sequence that is similar to the historical sequences of other variables. This means that there may be mutual dependencies between different variables at different time stamps, and that predicting the interdependent relationships between the historical and current time series is valuable.

Let $\{x^t\}, t \in \{1, 2, \dots, T\}$ denote an observed multivariate time series, where $x^t \in \mathbb{R}^n$ for some $n > 1$. Assume that x^t relies heavily on observations from time $t - d$ to $t - 1$ for some integer $d > 0$. So we define d as the lookback step size and a subject at time t to be denoted as $X^t = \{x^{t-d}, x^{t-d+1}, \dots, x^t\}$, where $x^{t-j} \in \mathbb{R}^n, j = 1, \dots, d$. The observation sequence of the i th variable in $\{x^t\}$ from time $t - d$ to t is denoted as $X_i^t = \{x_i^{t-d}, x_i^{t-d+1}, \dots, x_i^t\}$, where $X_i^t \in \mathbb{R}^{d+1}$. The objective is to predict the future sequence value x^{t+h} at time $t + h$ using the data collected up to time t , where h denotes a specified forward prediction step determined by the real-time series data and task context.

This study builds upon the related work on instance-level multivariate time series prediction by Xu, et al. in 2021^[29] and proposes an innovative IGMMA framework. The specific structure of the IGMMA framework is shown in Figure 1. The core idea of this framework is to blend current time series information with historical information from different variables and provide predictions by weighting various candidate neural network models using model averaging methods. This process involves the following steps:

- 1) Learn the training and mini-batch instances and convert them into embedding forms. To tackle the complexity and implementation difficulties of using only training examples, a

sampler is employed to select the most pertinent historical data and information by calculating similarities between each mini-batch and training instance. This part is contained in the three modules (a), (b), and (c) of Figure 1, of which we will elaborate on the specific details in Subsection 2.1.

- 2) Use the graph aggregation module to aggregate the information from the sampled training instances into mini-batch instances. The aggregated information, along with the mini-batch instances, is concatenated to obtain a feature representation. The arrows from modules (b) and (d) of Figure 1 converge at the square representing the ‘combine’ operation, illustrating the described process.
- 3) Select the generated features and construct models with different feature dimensions. An optimal linear estimation of the original and randomly selected features is obtained using the least squares method. The weights are then used to assign importance to each estimate. This strategy provides a balanced and compromised solution in terms of prediction effectiveness and model complexity, and it is demonstrated in module (e) of Figure 1.
- 4) Modify the loss function based on the Mallows criterion to make it more suitable for graph neural networks. This adjustment enhances the optimization process.

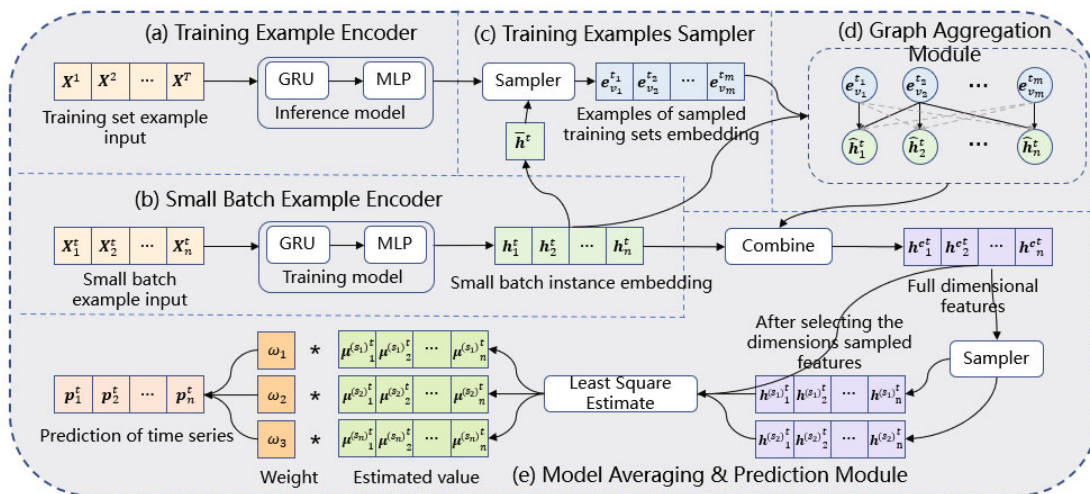


Figure 1 The overall architecture diagram of the IGMMA framework

The structure and notation definitions of this framework will be further elaborated in the following subsections.

2.1 Feature Extraction and Graph Module

The training instance encoder aims to learn the representations of sequence instances in the training set $\mathcal{D}_{train} = \{X^1, X^2, \dots, X^T\}$, where $X^i \in \mathbb{R}^{n \times (d+1)}$ and T represents the number of

timestamps in the training set. The features of the training instances in $\mathcal{D}_{train}$ are input into a network structure consisting of a Gated Recurrent Units (GRU) and a three-layer Multilayer Perceptron (MLP), obtaining the embeddings $E = \{E^1, E^2, \dots, E^T\}$, where $E^i \in \mathbb{R}^{n \times l}$ and l represents the dimension of the output of the MLP.

For each mini-batch $\mathcal{D}_{train}$, we input the features $X^t = \{X_1^t, X_2^t, \dots, X_n^t\}$ of n instances $\{v_1^t, v_2^t, \dots, v_n^t\}$ at time t into a network with the same structure as the training instance encoder, undergoing a similar feature extraction process. The output is the embedding $H^t = \{h_1^t, h_2^t, \dots, h_n^t\}$ of the mini-batch instances, where $h_i^t \in \mathbb{R}^l$.

To avoid the substantial computational burden of directly aggregating information from training instances to mini-batch instances, we use a training instance sampler to select the most relevant training instances D_{train}^{sample} for each mini-batch from D_{train} . Firstly, the mean $\bar{E} = \{\bar{E}^1, \bar{E}^2, \dots, \bar{E}^T\}$ of the embeddings of each timestamps in the training instances and the mean of the embeddings of the mini-batch instances $H^t = \{\bar{h}_1^t, \bar{h}_2^t, \dots, \bar{h}_n^t\}$ are calculated according to

$$\bar{E}^i = \frac{1}{n} \sum_{j=1}^n E_j^i, \quad \bar{h}^t = \frac{1}{n} \sum_{j=1}^n h_j^t,$$

where, $\bar{E}^i \in \mathbb{R}^l$ is the mean embedding of the training instances at time i , and $\bar{h}^t \in \mathbb{R}^l$ is the mean embedding of the mini-batch instances H^t . Subsequently, we calculate the cosine similarity between each timestamp's embedding $\bar{E}^i$ and $\bar{h}^t$ to find the most relevant instance to each mini-batch instance, and select k embeddings at timestamps that are closest to $\bar{h}^t$ based on similarity. Following this, the training instances within these k timestamps are sampled to create the sampled training instances with a total number $m = n \times k$.

In the graph neural network module, a bipartite graph is constructed to capture the mutual connection and dependence between the sampled training instances D_{train}^{sample} and the mini-batch instances D_{batch} . Given m sampled training instance embeddings $E_s = \{e_{v_1}^{t_1}, e_{v_2}^{t_2}, \dots, e_{v_m}^{t_m}\}$, and n mini-batch instance embeddings $H^t = \{h_1^t, h_2^t, \dots, h_n^t\}$, the information of the sampled training instances is aggregated with the mini-batch instances using cosine similarity between E_s and H^t as the initial aggregation weight:

$$A_{ij} = \text{Cosine}(h_i^t, e_{v_j}^{t_j}) = \frac{W_h h_i^t \cdot W_e e_{v_j}^{t_j}}{\|W_h h_i^t\| \cdot \|W_e e_{v_j}^{t_j}\|},$$

where W_h and W_e are two mapping matrices.

Due to the potential issues of over-smoothing in fully connected graphs, the aggregation weights A_{ij} from all training instances to the mini-batch instances v_i , where $j \in \{1, 2, \dots, m\}$, should retain only the top N weights and mask all other weights as 0. Finally, using the masked adjacency matrix $\hat{A}$, information is aggregated from the closest N training instances to each mini-batch instance:

$$\hat{h}_i^t = \frac{1}{\mathcal{N}_i} \sum_{j \in \mathcal{N}_i} \hat{A}_{ij} W_e e_{v_j}^{t_j},$$

where $\mathcal{N}_i$ is the set of top N closest training instances for the mini-batch instance v_i^t . The $\hat{h}_i^t$ is the information aggregation from the historical training instances to the instance v_i^t at timestamp t .

2.2 Model Averaging Module

The goal of the model averaging module is to enhance the efficiency of converting extracted feature information into final prediction results. This is achieved by combining the least squares estimates of various features to obtain the outcome. This approach is known to enhance the accuracy and robustness of the model, particularly in situations where overfitting may occur. Weighted averaging, simple averaging, and voting are methods that are commonly used in model averaging. In this study, model averaging module employs the weighted averaging method.

We combine the information from the graph aggregation module with small batch instances to jointly make predictions about future time series. The aggregated information obtained from the training set instances, $\hat{h}_i^t$, serves as the first half, and the small batch instances, h_i^t , serve as the second half of the filling. By connecting these two halves, we obtain the concatenated feature vector $h_i^{c^t} = \text{Concat}(\hat{h}_i^t, h_i^t)$, where *Concat* denotes the concatenation operation of two vectors. Since both $\hat{h}_i^t$ and h_i^t are l -dimensional vectors, i.e., $\hat{h}_i^t, h_i^t \in \mathbb{R}^l$, the feature vector $h_i^{c^t}$ becomes a $2l$ -dimensional vector. This vector captures the feature information obtained from aggregating the current small batch instances and the training set instances up to the current point in the network structure.

2.2.1 Model Averaging Estimator

Following the idea of model averaging, let S denote the total number of candidate models. We randomly select $\frac{p_s}{2}$ features from each of the two components $\hat{h}_i^t$ and h_i^t to obtain various features corresponding to different p_s by combining them. Here, p_s represents the scale of the s th candidate model in the model averaging process, where $1 \leq s \leq S$. We denote the sampled features as $h^{(s)}$.

For various features $h^{(s)}$, which correspond to models with different numbers of dimensions and randomly selected dimensions, we perform weighted averaging of the models to increase the diversity of model features. Let $\hat{\mu}^{(s)} = h^{(s)}\hat{\beta}^{(s)}$ be the estimation based on the s th candidate model, where $\hat{\beta}^{(s)}$ is the parameter fitted using the least squares estimation. Since it is strictly required that no future information should be used during model training, and no label values should be used as feature information to participate in the training process, according to the least squares estimation, we obtain the current small batch of sequence instances $v^t = \{v_1^t, v_2^t, \dots, v_n^t\}$ under all variables, where $v^t \in \mathbb{R}^n$, and slice its features $X^t = \{x^{t-d}, x^{t-d+1}, \dots, x^t\}$, $X^t \in (d+1) \times n$, to obtain the observation value x^t at the last timestamp t . Further, let Y be the matrix stacked according to the timestamp, i.e., $Y = \{x^{d+h-1}, x^{d+h}, \dots, x^T\}$. At this point, the calculation method of the estimated parameter is $\hat{\beta}^{(s)} = (h^{(s)'}h^{(s)})^-h^{(s)'}Y$, where A^- and A' represent the generalized inverse and transpose of matrix A , respectively.

In the process of defining the network model based on model averaging, in addition to the trainable parameters defined in the GRU, multilayer MLP, and other linear layers, an additional

vector is added that needs to participate in gradient updates. Its representation is shown as

$$w = (w_1, w_2, \dots, w_S)' \in \mathcal{W} \triangleq \left\{ w \in [0, 1]^S : \sum_{s=1}^S w_s = 1 \right\}. \quad (1)$$

Then the corresponding model averaging estimator is

$$\hat{\mu}(w) = \sum_{s=1}^S w_s \hat{\mu}^{(s)}.$$

Finally, the obtained output $\mathbf{p}^t = (\mathbf{p}_1^t, \dots, \mathbf{p}_n^t)'$ is the predicted value of the time series, and its calculation method is shown as follows:

$$\mathbf{p}^t = \sum_{s=1}^S w_s h^{(s)} \hat{\beta}^{(s)} = \sum_{s=1}^S w_s P^{(s)} Y = P(w) Y,$$

where $P^{(s)} = h^{(s)} (h^{(s)})' h^{(s)} \text{ }^{-1} h^{(s)'} \text{ }$ and $P(w) = \sum_{s=1}^S w_s P^{(s)}$.

2.2.2 Mallows Type Model Averaging Criterion

Based on the model averaging module proposed in the previous section, it is employed to replace the conventional approach of utilizing linear layers for dimensionality reduction and mapping high-dimensional features to the final output in deep learning methods. However, solely relying on basic loss terms such as mean absolute error (MAE), mean squared error (MSE), and the summation of L_2 regularization terms as the model's loss function cannot accurately represent the optimization objective. Although the L_2 norm scales all parameters in the model by predetermined weight decay coefficients to prevent excessively large parameter values and constrain the model's size and complexity, it uniformly penalizes all parameters and lacks targeted penalization for relatively large-scale candidate models. Consequently, the optimization outcome tends to favor larger models. Hence, customization of the loss function is imperative to better align the optimization objective with the Mallows criterion.

Qiu, et al.^[28] applied the Mallows-type model averaging method to machine learning. The weight vector is estimated by minimizing the Mallows criterion, which is defined as follows:

$$C_n(w) = \|Y - P(w)Y\|^2 + 2\hat{\sigma}^2(w) \text{tr}[P(w)],$$

where $\hat{\sigma}^2(w)$ is the estimator of the mean error variance, Y is the data vector corresponding to the response variable, and $\text{tr}(A)$ represents the trace of matrix A .

To make the loss function more consistent with the actual module we introduced and this deep learning multivariate time series prediction task, the loss function based on the Mallows criterion is defined as follows:

$$\mathcal{L}' = \frac{\sum_{v_i^t \in \mathcal{D}_{batch}} (\mathbf{p}_i^t - \mathbf{x}_i^{t+h})^2}{|\mathcal{D}_{batch}|} + 2 \frac{\hat{\sigma}^2(w) \text{tr}[P(w)]}{|\mathcal{D}_{batch}|} + \lambda \|\Theta\|_2^2. \quad (2)$$

Here, $\mathcal{D}_{batch}$ is the sequence instances in each small batch, and $\mathbf{p}_i^t$ and $\mathbf{x}_i^{t+h}$ are the predicted values at time t and the real value of the variables v_i at time $t+h$, respectively. In the penalty

term, λ represents the regularization coefficients, and Θ represents all the parameters in the model except w .

The overall structure of the loss function consists of a loss term and a penalty term. The loss term uses MSE to penalize prediction errors at the squared level, providing a sensitive measure of model prediction deviation. The penalty term includes penalties for the size of the candidate model and all parameters of the model. The penalty for the parameter sizes of the model employs L_2 regularization to constrain overall parameter magnitudes.

In model averaging based on the Mallows criterion, the trace of the matrix is commonly used to characterize model complexity. It estimates the rank of the covariance matrix, which determines the number of free parameters in the model. The loss function combines the trace of the matrix with the sum of squared residuals, weighing model complexity and goodness of fit. This approach selects a relatively simple model with good explanatory power. Introducing a penalty term for candidate model complexity in the loss function can help address overfitting or underfitting problems, thereby enhancing the model's predictive accuracy.

The loss function, as shown in (2), is minimized to find the optimal weight vector based on the Mallows criterion. The weight vector w remains subject to the constraints described in (1). In PyTorch, the loss function is typically defined as part of the model for calculating and optimizing the loss during training. However, when modifying the built-in loss function to enhance model performance, it is necessary to customize the loss function to align it closely with the intended optimization target. This is achieved by creating the MallowLoss class, inheriting from the base nn.Module, and implementing the forward() method to calculate custom Mallows loss values. The method takes multiple input tensors (usually observed value y and predicted value $\hat{y}$), measures the error degree between them using established methods, and returns a scalar representing the loss value. The MallowLoss class defined in this paper accepts $P(w)$ as an additional parameter for loss value calculation. $P(w)$ is one of the output values obtained from the model's forward propagation, which is then passed to the MallowLoss instance to compute the user-defined Mallows loss.

2.3 Algorithm of IGMMA Framework

We employed a small-batch stochastic gradient descent algorithm to train the model while utilizing the Adam algorithm^[30] to adjust the learning rate.

To ensure that the weight of each model align with common sense, we incorporated the WeightConstraint class during training. Within this class, we screened the parameters involved in gradient updates based on their logical definitions. Specifically, we iterated over the model's named parameters and applied constraints to the weight vector w . We constrained its components to fall within the range of 0 and 1, and then normalized the vector. This resulted in constraints where each component is greater than 0 and less than 1, and the sum of the components equals 1. In each iteration, we enforced this constraint, effectively treating the problem as a constrained optimization problem. Algorithm 1 illustrates the training process.

Algorithm 1 IGMMA framework algorithm**Input:** Training data $\mathcal{D}_{train}$, initial parameter Θ , and learning rate γ **Output:** Time series prediction $\mathbf{p}^t$

```

1: while The algorithm termination criteria have not been met do
2:   The features of the instance in  $\mathcal{D}_{train}$  are encoded as training instance embeddings  $E$ .
3:   for  $\mathcal{D}_{batch}$  in  $\mathcal{D}_{train}$  do
4:     Encode the characteristics of the small-batch  $\mathcal{D}_{batch}$  instance as embeddings  $H_t$ .
5:     Sample the most relevant training instance  $\mathcal{D}_{train}^{sample}$  from  $\mathcal{D}_{train}$  for  $\mathcal{D}_{batch}$ .
6:     The information in  $\mathcal{D}_{train}^{sample}$  is aggregated into  $\mathcal{D}_{batch}$  to form  $h^c = Concat(\hat{h}^t, h^t)$ .
7:      $\hat{h}^t$  and  $h^t$  are randomly sampled with the same dimension to obtain  $h^c$ .
8:     Using  $h^c$  to calculate its corresponding  $P^{(s)}$  matrix for parameter estimation, several
       least squares estimates are obtained.
9:     The calculation results are weighted by  $w$ .
10:    Predict future time series  $\mathbf{p}^t$ .
11:    The random gradient of parameter  $\Theta$  is calculated according to the formula method.
12:    Update parameter  $\Theta$  according to gradient and learning rate  $\gamma$ .
13:  end for
14: end while

```

3 Empirical Analysis

In the previous section, we built upon the model averaging method by modifying the loss function to achieve better prediction performance. We defined an instance-level model averaging framework based on the Mallows criterion and graph neural network model. We plan to conduct empirical research on a multivariety futures dataset using this framework. In addition, we defined the instance-wise graph-based model averaging (IGMA) framework, which only incorporates the model averaging module without modifying the loss function.

Ablation experiments were conducted to demonstrate the effectiveness of the model averaging module. We mainly implemented IGMA, IGMMA, and other frameworks proposed in the literature based on PyTorch 1.11.0^[31]. We also reproduced other necessary baseline models based on this deep learning framework. All experiments were conducted using a single NVIDIA GeForce RTX 3090 graphics card. The software required for the experiments included the programming language Python 3.9.12, data processing libraries NumPy 1.24.2 and Pandas 1.4.2, machine learning framework scikit-learn 1.1.0, deep learning frameworks PyTorch 1.11.0 and TensorFlow 2.8.0, CUDA 11.6/cuDNN 8.4.0, and hyperparameter optimization framework Ray 2.3.0^[32].

3.1 Multivariate Future Time Series Data Collection

Futures trading is a derivative trading method that mainly involves the trading of future commodities or financial assets. This trading method can be broadly classified into two categories: Commodity futures and financial futures. Commodity futures are typically divided into three major categories: Agricultural, metal, and energy. Financial futures mainly consist of

foreign exchange futures, interest rate futures, and stock index futures.

To ensure data accuracy, consistency, and completeness, we collected data from several sources and cross-validated (including unified format conversion) historical and real-time data from various data providers. The primary data flow is shown in Figure 2. Broker in Figure 2 refers to a company or an individual that acts as an intermediary between traders and the exchange, while DataVendor represents a data provider.

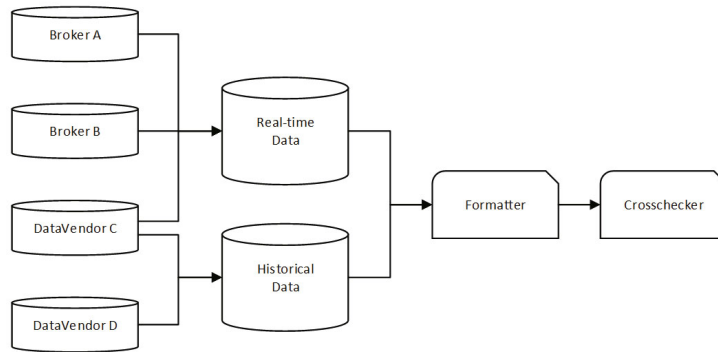


Figure 2 The main flow direction of the data

Cross-validation among several data sources can be achieved through mutual support, supplementation, and correction of each data source, thus providing more accurate, consistent, and comprehensive information. At the same time, because of the differences in data source links and processing methods, speed comparisons can be made on real-time data to improve timeliness. For historical and real-time futures data, we compared the above data providers in terms of data quality, required workload, cost, etc., and we selected Tonghuashun (iFind) and JQData as historical data sources based on their data interface and cost suitability. For real-time data, we accessed real-time data and minute data integration from Macro Source Futures and Nanhua Futures through the Comprehensive Transaction Platform (CTP). Algorithm 2 outlines the concatenation method, where tick data refers to tick-by-tick data, and bar data refers to candlestick data generated from high-frequency tick data for clearer display and easier storage of information. Here *open*, *close*, *high*, *low*, *volume*, and *amount* represent opening price, closing price, highest price, lowest price, trading volume, and trading value, respectively.

In this study, when we select a variety of futures data, we need to consider its market liquidity, trading activity, and the characteristics of a longer listing time, so we excluded some varieties such as No.2 Soybean, Rapeseed, Pumai, Wire, and other varieties with poor liquidity and inactive trading. Thus, forty-one futures varieties were finally determined: No.1 Soybean, Soybean Meal, Rapeseed Meal, Soybean Oil, Rapeseed Oil, Palm Oil, Sugar, Egg, Corn, Corn Starch, Cotton, Natural Rubber, Flat Glass, Methanol, Polyethylene (PP), Polypropylene (PE), Polyvinyl Chloride (PVC), Pure Terephthalic Acid (PTA), Bitumen (BU), Ferrosilicon, Manganese Silicon, Fuel Oil, Rebar, Iron Ore, Coke, Coking Coal, Hot Rolled Coils, Crude Oil, Silver, Gold, Aluminum, Copper, Zinc, Nickel, Lead, Tin, Shanghai Stock Exchange 50, China Securities Index 300, China Securities 500, 5-year Treasury bonds, 10-year Treasury bonds.

The dataset used in this paper contains the price data for the above futures contracts for the period from January 1, 2015, to January 28, 2022, and the data sampling frequency is 5 min. The data were collected according to the method described in the previous section after the database was extracted as a comma-separated values file (comma-separated values, CSV) and then read.

Algorithm 2 Tick data concatenation bar data algorithm

Input: tick_data, last_volume, last_amount

Output: open, close, high, low, volume, amount

```

1: open, close, high, low, volume, amount=0;
2: while min in cur_min do
3:   if open=0 then
4:     open=tick;
5:   end if
6:   if high=0 or high<tick then
7:     high=tick;
8:   end if
9:   if low=0 or low>tick then
10:    low=tick;
11:  end if
12: close=tick; volume+=last_volume; amount+=last_amount;
13: end while

```

We take the multivariate futures dataset in Section 3 as an example. Using $d = 40$, a variable dimension of $n = 41$, and a prediction step of $h = 1$, we obtain the price series or numerical matrix of the past 40 days for the forty-one futures at time t . This information allows us to predict the price cross-section information of the forty-one futures at a future time. Thus, the problem involves a continuous rolling window approach to forecasting. The basic statistical information for the dataset is shown in Table 1.

Table 1 Statistical information of dataset

Data set	Time	Variables	Sampling frequency
modified_futures_5 min	204792	41	5 min

We split this dataset into the training set (60%), verification set (20%), and test set (20%) in strict order of the timestamps. All models uniformly use the same lookback step and forecasting step size. The input feature lookback step size for each instance window is 40, i.e., the input value at each time is the time series of the past 40 timestamps for each variable, and the prediction step horizon is 1, i.e., the value at the next timestamp.

3.2 Comparison Experiments of IGMTF and IGMA

To validate the effectiveness of the model averaging module, this experiment compares and analyzes two scenarios: The IGMA framework (IGMTF hyperparameter variant with the

model averaging module) and the IGMTF (IGMTF hyperparameter variant without the model averaging module). The objective is to investigate the impact of the model averaging module on model performance. Specifically, the IGMTF hyperparameter variant framework serves as a reference, and it includes cases in which the hidden dimensions of the training instance and the small-batch instance GRU layer are set to 8, 32, 64, 128, and 256. Consequently, the total number of feature dimensions are 16, 64, 128, 256, and 512.

To ensure consistent empirical analysis, as described in Section 2, it is essential to maintain the same hyperparameter settings except for the aforementioned variations. Specifically, we use a batch size of 64, a learning rate of 0.0001, and employ an Adam optimizer for 50 training iterations. This ensures that only the specified variables change, which lends credibility to the conclusions drawn. The model with the smallest relative square root error (RRSE) on the validation set is considered to be the optimal model, and its RRSE, relative absolute error (RAE), and correlation coefficient (CORR) metrics are evaluated on the test set to assess the performance of the model beyond the training samples.

We apply the RRSE, RAE, and CORR as evaluation indicators to evaluate the performance of each model on the validation and testing sets of the futures benchmark dataset. The “↑” at the end of the evaluation indicator indicates that the higher the value, the better the overall performance of the model. If the arrow is shown as “↓”, the value decreases with better performance of the model. IGMTF_512D, IGMTF_256D, IGMTF_128D, IGMTF_64D, and IGMTF_16D refer to the scenarios where the hidden layer dimensions of the IGMTF framework training instance and the GRU layer in small batch instances are 256, 128, 64, 32, and 8, respectively. Therefore, the dimensions of the feature vectors $\text{Concat}(\hat{h}^t, h^t)$ are input into the linear layer before the prediction results correspond to 512, 256, 128, 64, and 16 are obtained. In the IGMA framework defined in this article, except for the model averaging module, the hyperparameters follow the IGMTF framework setting with a total feature vector dimension of 16. Table 2 shows the performance results under the above assessment indicators for each framework. A paired t-test was carried out between IGMA and a reference method at a significance level of 0.05, and * is appended when the enhancement of IGMA over the method achieves statistical significance. The numbers in bold indicate the better result in the comparison.

Table 2 Forecasting results of multivariate time-series models

Model	RRSE (↓)	RAE (↓)	CORR (↑)
IGMTF_16D	*6.853 × 10 ⁻³	*5.432 × 10 ⁻³	0.999808
IGMTF_64D	*6.321 × 10 ⁻³	*4.662 × 10 ⁻³	0.999811
IGMTF_128D	*6.187 × 10 ⁻³	*5.495 × 10 ⁻³	0.999842
IGMTF_256D	*6.042 × 10 ⁻³	*4.170 × 10 ⁻³	0.999848
IGMTF_512D	*6.029 × 10 ⁻³	*3.509 × 10 ⁻³	0.999854
IGMA	5.491 × 10⁻³	2.888 × 10⁻³	0.999881

To make the results more intuitive and easier to compare, the RRSE, RAE, and CORR of the test set in Table 2 were extracted as the data of the stacked line bar chart, and the predicted results are displayed in Figure 3. Among the results, the RRSE and RAE indicators

correspond to the left coordinate axis in Figure 3, and the CORR indicators corresponds to the right coordinate axis.

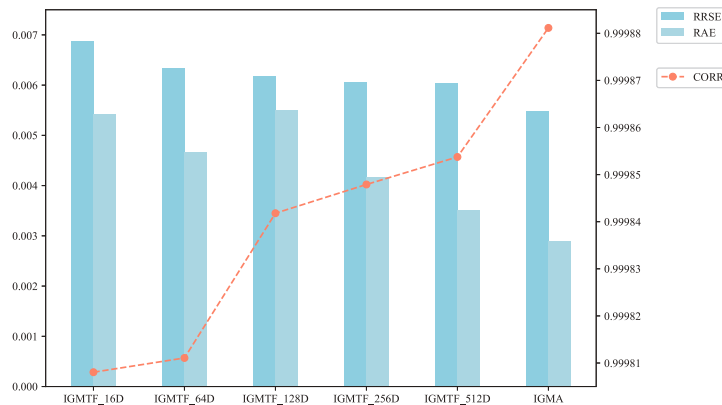


Figure 3 Visualization of prediction results of multivariate time series model

From the chart, we can infer that the prediction performance (measured by the RRSE, RAE, and CORR indicators) improves as the hidden feature dimension increases within the IGMTF framework. It can be concluded that the total feature dimension of the IGMTF framework is a hyperparameter that exhibits sensitivity within the range of 16 to 512 dimensions. Moreover, higher hidden feature dimensions allow for the capture of more information, thereby enhancing the predictive capability.

By augmenting the IGMTF framework with the model averaging module, the IGMMA framework achieves the best performance of all frameworks, as evidenced by its superior performance on the three indices. This ablation analysis of the individual module reveals that the model averaging module significantly enhances the performance of the IGMMA framework on the test set, reducing the RRSE to 0.005491, the RAE to 0.002888, and yielding a CORR value of 0.999881. Compared with the prediction performance of IGMTF_16D, these results show a significant 19.87% reduction in RRSE and a 46.83% decrease in RAE. Because the CORR has a high baseline value, its increase ratio is not discussed. Compared with the IGMTF_512D framework, which closely follows the IGMMA framework, the same indices exhibit a respective reduction in error of 8.92% and 17.70%. In other words, the predictive performance of IGMMA, achieved by incorporating the model averaging module in IGMTF, outperforms that of IGMTF_512D, indicating that IGMMA differs from the original structure and not only generates feature dimensions.

We also compared the time required for each iteration round of the IGMTF with different dimensions, as shown in Table 3. The comparison of the training time for each model is analyzed. When the hidden layer dimension is small, the training time increases with an increase in dimensions, but the change range is small. When the hidden layer dimension reaches 256 dimensions, i.e., the total feature dimension is 512, the duration of each round of iteration increases significantly from 70 s to 85 s to 120 s with the total feature dimension of the model increasing from 16 to 128 to 512, and the time is not proportional to the dimension scale but

is positively correlated.

Table 3 Training time of multivariate time-series models

	IGMTF_16D	IGMTF_64D	IGMTF_128D	IGMTF_256D	IGMTF_512D	IGMA
Time	~70 s	~72 s	~80 s	~85 s	~120 s	~85 s

Therefore, its predictive performance and training time are basically positively correlated with the hidden layer dimension, i.e., the parameter size of the model. As a result, the IGMA framework performs better on both the verification and the test sets than other IGMTF models with settings for the hidden layer dimension. That is, when the model averaging module is added, a relatively small number of parameters can be used to achieve greater prediction accuracy. Thus, the model averaging module added to IGMTF significantly improves the performance of multivariate time series forecasting.

3.3 Comparison Experiments with IGMMA and Discussion

We will compare the proposed IGMMA framework with the multivariate time series prediction method as follows:

- 1) LSTNet: A deep neural network architecture framework for modeling long- and short-term time series patterns. It was proposed by Lai, et al.^[33]. It uses CNN to extract short-term local dependency patterns, whereas RNN is used to capture complex long-term dependency relationships.
- 2) TCN: A convolutional neural network architecture framework proposed by Bai, et al.^[34]. It uses a series of expanded convolutional layers and residual blocks to construct the network structure, which can expand the receptive field of the convolutional kernel without increasing the number of parameters. This enables the TCN to better capture long-term dependencies in sequences.
- 3) LightCTS: A prediction framework for correlated time series (CTS) proposed by Lai, et al.^[35]. It aims to explore a different direction of implementing more efficient and lightweight models that can maintain accuracy while being deployed on resource-constrained devices. The framework adopts pure time and space operation stacking instead of alternating stacking with higher computational costs.
- 4) IGMA framework: A framework that only adds the model averaging module without modifying the loss function to explore the impact of modifying the calculation method of the target function to be optimized on the model's prediction performance.

For TCN, LSTNet, and LightCTS frameworks, we chose to retain most parameter settings in the original open-source code. For the lookback and prediction step sizes, all models were uniformly set to 40 and 1, while the other parameters remained default settings (if there are obvious hyperparameters that need to be modified to conform to the dataset structure, they should be modified accordingly).

Using the same preprocessing method for multivariate future datasets, we replicated the baseline model based on the corresponding open-source code available on GitHub. To evaluate the performance on the validation set, we employed Ray to search the framework's hyperparameters to identify the optimal combination. The parameters involved in the grid search included the learning rate (0.0001, 0.0005, 0.001), the number of samples per batch for small-batch instances (non-training instances) (16, 32, 64), and the weight decay coefficient, which represents the coefficient of the L_2 regularization term in $\mathcal{L}'$ (0.0001, 0.001). This yielded 18 hyperparameter combinations. Because LSTNet and TCN models exhibited exceptionally short training times per iteration, we set their iteration rounds to 100. For the other models (LightCTS, IGMA, IGMMA), the number of iteration rounds was set to 50, and the final result was obtained by averaging the outcomes of five experiments.

We also applied the same indicators, namely RRSE, RAE, and CORR to evaluate the performance of each model on the validation and test set of the futures dataset. Similarly, the value “↑” beside the evaluation indicator means that the higher the value, the better the overall performance of the model; if the arrow is shown as “↓” the value decreases as the model performance increases. Table 4 shows the performance results under the above assessment indicators in each framework.

Table 4 Forecasting results of multivariate time-series models

Model	RRSE (↓)	RAE (↓)	CORR (↑)
LSTNet	$*9.800 \times 10^{-3}$	$*6.220 \times 10^{-3}$	0.999604
TCN	$*6.126 \times 10^{-3}$	$*3.656 \times 10^{-3}$	0.999843
LightCTS	$*6.560 \times 10^{-3}$	$*4.327 \times 10^{-3}$	0.999809
IGMA	5.491×10^{-3}	2.888×10^{-3}	0.999881
IGMMA	4.951×10^{-3}	2.627×10^{-3}	0.999885

From Table 4, we can conclude that IGMMA framework outperforms all other frameworks on the three indices of the test set by incorporating the model averaging module and modifying the loss function based on IGMTF. The IGMMA framework achieves a low RRSE on the test set of 0.004951, an RAE of 0.002627, and a CORR of 0.999885. The prediction performance of IGMMA framework is slightly better than that of IGMA framework proposed earlier, leading to a decrease of 9.83% and 9.04% in RRSE and RAE, respectively. In other words, optimizing the modified loss function can enhance the prediction effectiveness. IGMMA, which incorporates the model averaging module, outperforms all the other models used for comparison in terms of prediction accuracy and generalization performance.

The RRSE, RAE, and CORR of the test set in Table 4 were visualized using a stacked line bar chart, and the results are shown in Figure 4. Observing the prediction results chart, it becomes clear that LSTNet's performance on this future dataset deviates significantly from that of other time series prediction frameworks. This divergence can be attributed to LSTNet's model structure, which consists of relatively simplistic CNNs and RNNs. Such a structure may impose limitations on its performance when it encounters time series data that contain long-term dependencies. Insufficient feature extraction by CNNs may also impede the comprehensive representation of the inherent complexity and patterns during subsequent modeling. In

comparison to LSTNet, TCN demonstrates slightly superior prediction capabilities. However, similar to LSTNet, it may encounter slight challenges due to CNN feature extraction problems. LightCTS, which stands out as a novel model with outstanding empirical performance and moderate complexity, surpasses only LSTNet in terms of effectiveness. One possible reason for this disparity may be the modeling of CTS. In general, although objective correlations exist between specific objects in the multivariate futures price series, these correlations tend to attenuate over a long-term series. Conversely, the IGMA framework enhances its alignment with the dynamics of multivariate futures prices by extracting the most relevant information from historical series through sampling and aggregation techniques.

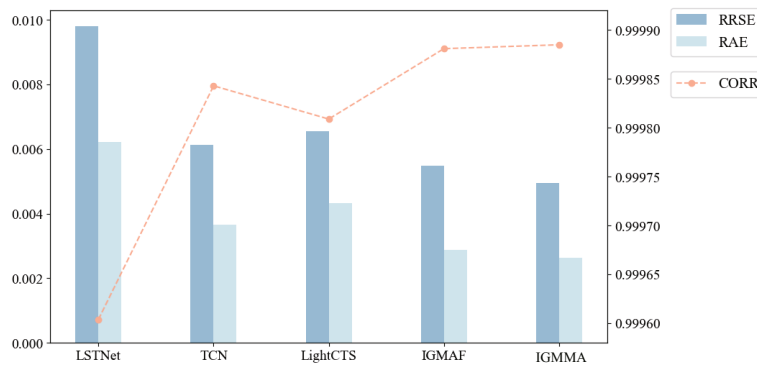


Figure 4 Visualization of prediction results of multivariate time series model

The IGMMA framework presents innovative enhancements to the widely employed linear layer in deep learning methods. It replaces this layer with the model averaging approach and includes a more suitable penalty term for selecting model weight parameters. By modifying the Mallows criterion, the model achieves substantial improvements in prediction accuracy and generalization performance.

We also conducted an empirical study to analyze the time required for each iteration of different multivariate time series forecasting methods, as presented in Table 5. According to Table 5, there is generally no clear and absolute positive or negative correlation between the performance and training time of the various models. We can make the following observations for each type of model (except IGMA):

- 1) The LSTNet and TCN models exhibited the shortest time, approximately 10 seconds and 20 seconds, respectively, which indicates relatively high training speeds. However, as mentioned earlier, their performance was relatively poor, which could be attributed to insufficient model complexity and inadequate feature extraction.
- 2) The LightCTS model required the longest time per epoch, around 130 seconds, suggesting a relatively slow training process. This model performed relatively well on the test set from the experimental results, indicating certain advantages for time series data processing.

- 3) The IGMMA model took about 120 seconds for each epoch, longer than IGMA, indicating that modifying the loss function increased the duration required for each epoch. In contrast to IGMTF_512D and LightCTS, where each iteration round lasted around 120 seconds, the overall training time for deep learning tasks remained moderate. Given the relatively low parameter count, achieving a balance between accuracy and training time is crucial based on practical application needs. In addition, modifying the hyperparameters could make the model more suitable for the real-world task at hand.

Table 5 Training time of multivariate time series models

	LSTNet	TCN	LightCTS	IGMA	IGMMA
Time	~10 s	~20 s	~130 s	~85 s	~120 s

In summary, our proposed IGMMA framework significantly outperformed other baseline models in predicting accuracy on validation and test sets. It is evident that for multivariate time series prediction, integrating the model averaging module and adjusting the Mallows model averaging criterion with practical problems can greatly enhance the predictive power of the framework.

3.4 Hyperparameter Analysis

For deep learning tasks, the choice of hyperparameters is crucial as it significantly affects the performance of the model. In this section, we conduct a systematic experimental test to analyze the sensitivity and specific mode of action of the model's hyperparameters.

Some hyperparameters in the model, including the hidden layer dimension of the GRU+MLP layer feature extraction module, the number of time stamps sampled from the training instance, and the number of selected nearest neighbors in the graph aggregation module, etc., play vital roles in the experimental setup. It is important to ensure a fair comparison between the proposed IGMA framework and the baseline model by maintaining fixed variables. Therefore, in this section, we focus on three specific hyperparameters for empirical analysis: The learning rate of the model, the batch size of the samples per batch, and the weight decay coefficient (L_2 regularization term coefficient). The learning rate is set to $\{0.0001, 0.0005, 0.001\}$, the number of samples per batch is set to $\{16, 32, 64\}$, and the weight decay coefficient is set to $\{0.0001, 0.001\}$. Figures 5–7 illustrate the RRSE, RAE, and CORR under different hyperparameter settings.

The RRSE and RAE trends in Figures 5–7 exhibit similarity but differ from the CORR trends, aligning with the definition of the evaluation indicators. From the visualization of the error measurement indexes in Figures 5 and 6, it can be observed that the values of RRSE and RAE typically exhibit an increasing trend as the number of samples in each batch gradually increases from 16 to 64, except when the value of the learning rate is 0.0005. In fact, a smaller number of batch samples with a larger number of batch samples leads to a smaller error value of the model due to improved stability of the gradient calculation, smoother error curve during training, and reduced training time. Nonetheless, a smaller batch size may impact the model's generalization and introduce noise, potentially deviating the solution from the global optimum.

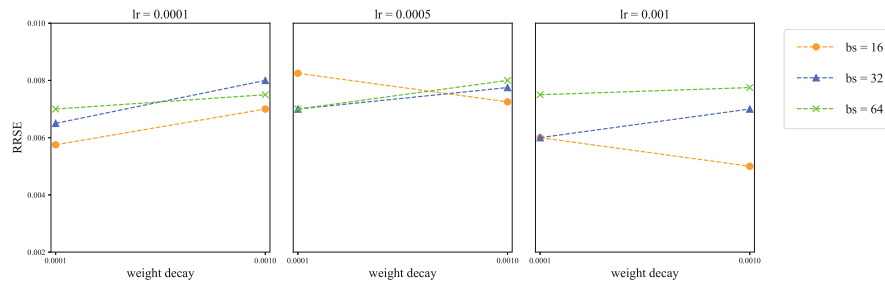


Figure 5 Relative square root error RRSE under hyperparameter adjustment

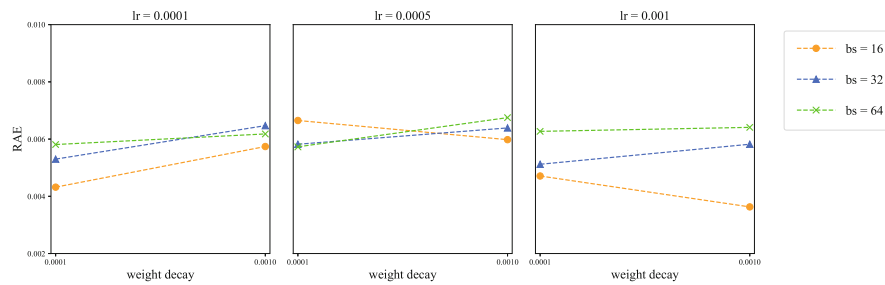


Figure 6 Relative absolute error RAE under hyperparameter adjustment

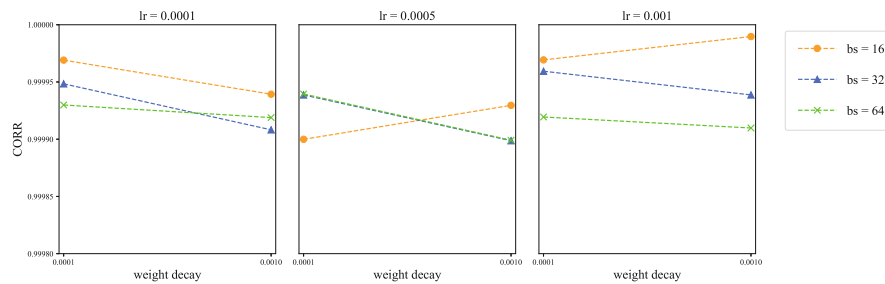


Figure 7 The correlation coefficient CORR under hyperparameter regulation

According to the experimental results, when the number of samples per batch is 16 and the learning rate is relatively high, specifically 0.0005 and 0.001, increasing the weight decay coefficient from 0.0001 to 0.001 significantly reduces the error value. However, when the learning rate is low and the number of samples per batch is relatively large, increasing the weight decay coefficient can increase the values of RRSE and RAE. This phenomenon may be attributed to the fact that, in the latter case, the higher weight decay coefficient overly penalizes the model parameters, thereby constraining the model's learning capacity.

When the number of samples per batch are 32 and 64, the value of RRSE increases with the weight decay coefficient increasing from 0.0001 to 0.0010. When the batch size is relatively large, specifically with 64 samples per batch, increasing the learning rate does not significantly affect the RRSE value. However, as the batch size decreases, particularly when the number of

samples per batch is relatively small, the RRSE shows a significant decrease. Using a larger batch size enhances the stability and robustness of the model during the training process, thereby reducing its sensitivity to changes in the hyperparameters. When the learning rate and the number of samples per batch are fixed, the effect of the weight decay coefficient on RRSE and RAE depends on the specific training situation and experimental results. In general, a higher weight decay coefficient imposes a stronger penalty on network structures with high complexity. However, an excessively high coefficient can lead to the underfitting of the model.

The analysis method for the CORR is similar to that of the RRSE and RAE, but it exhibits the opposite trend.

Figure 8 provides a summary of the time required for each iteration round and the classification of 18 hyperparameter combinations into 6 categories, considering batch sample sizes of 16, 32, and 64. The time required for each iteration round varies due to the different settings of the learning rate and weight attenuation coefficient. The top line in Figure 8 represents the time required when the batch sample size is 16, which is approximately 350 seconds, reaching its highest point when both the learning rate and weight attenuation coefficient are set to 0.0001. The bottom line represents the time required when the batch sample size is 64, which is approximately 150 seconds. The shortest time is when the learning rate and weight attenuation coefficient are set to 0.001. It is worth noting that additional variable calculations are required for hyperparameter adjustment using Ray, which slightly increases the time required for a model with the same parameter settings in the hyperparameter experiment compared with the data obtained in the main results section of the experiment. Although the specific position order of the corresponding points for each hyperparameter combination is determined by the specific parameter setting, the general trend is that the batch sample size increases from small to large, and the required time increases from short to long. Therefore, when the batch sample size is increased from 16 to 64, the time required for each iteration round significantly decreases. Considering both the model's forecasting ability and training time, the change in batch sample size results in significant fluctuations in the corresponding index. Thus, the batch sample size is considered to be a sensitive hyperparameter.

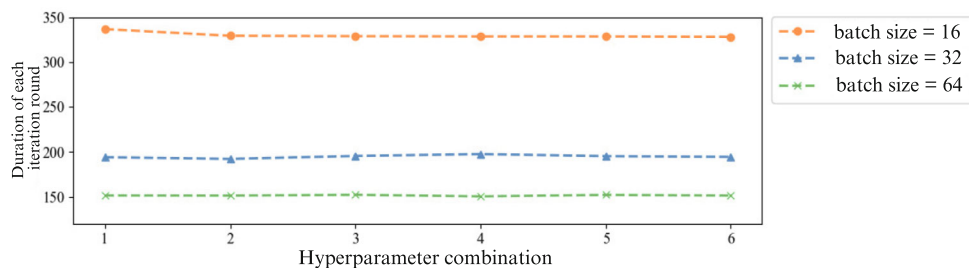


Figure 8 Duration of each iteration round for each hyperparameter combination

In summary, we determined the optimal hyperparameter combination for the sample number per batch, learning rate, and weight decay as $\{16, 0.001, 0.001\}$. However, in practical applications, particularly when forecasting with real-time future data, it becomes necessary to consider

the training and forecasting time of the model. Therefore, trade-offs and compromises should be made between prediction accuracy and factors such as time, resources, and manpower.

4 Conclusions

We have introduced an innovative approach that combines model averaging with deep learning methods. We distinguish it from traditional model averaging techniques. The model averaging module is an integral component of the IGMA network and is not treated as a separate entity. Considering the characteristics and preferences of the MMA, a new loss function was defined for optimization based on Mallows criterion, and empirical research was conducted on the proposed framework algorithms.

The experimental results demonstrated that the model averaging module achieves higher prediction accuracy with a relatively small parameter scale and better the performance of the original model on the multivariate futures dataset. The IGMMA framework, which incorporates the model averaging modules and modifies the loss function, outperforms all other baseline models on the test set, despite having a slightly higher model complexity. In conclusion, the IGMMA framework effectively addresses the discussed multivariate future time series prediction problem, showcasing the potential application of model averaging in deep learning.

In future studies, we aim to widen the scope of multivariate time series prediction beyond the limited use of price index series for various futures varieties under a specific timestamp. We plan to enhance the analysis to encompass multi-feature and multivariate scenarios by incorporating multidimensional information, including but not limited to trading volume. Furthermore, we intend to explore the integration of alternative data sources, such as public opinion data, to address multivariate time series forecasting challenges without significantly increasing model complexity or deviating from real-world conditions.

Conflict of Interest

The authors declare no conflict of interest.

References

- [1] Li Y, Yu R, Shahabi C, et al., Diffusion convolutional recurrent neural network: Data-driven traffic forecasting, *International Conference on Learning Representations (ICLR)*, Vancouver, 2018.
- [2] Wu Y, Liu Y, Ahmed S H, et al., Dominant data set selection algorithms for electricity consumption time-series data analysis based on affine transformation, *IEEE Internet of Things Journal*, 2020, **7**(5): 4347–4360.
- [3] Cheng D, Yang F, Xiang S, et al., Financial time series forecasting with multi-modality graph neural network, *Pattern Recognition*, 2022, **121**: 108218.

- [4] Yu L and Gao X, Improve robustness and accuracy of deep neural network with $L_{2,\infty}$ normalization, *Journal of Systems Science & Complexity*, 2023, **36**(1): 3–28.
- [5] Wang C and Gao Q, High and low prices prediction of soybean futures with LSTM neural network, 2018 *IEEE 9th International Conference on Software Engineering and Service Science (ICSESS)*, Beijing, 2018, 140–143.
- [6] Sun T, Wang J, Ni J, et al., Predicting futures market movement using deep neural networks, 2019 *18th IEEE International Conference on Machine Learning and Applications (ICMLA)*, Boca Raton, 2019, 118–125.
- [7] Ji L, Zou Y, He K, et al., Carbon futures price forecasting based with ARIMA-CNN-LSTM model, *Procedia Computer Science*, 2019, **162**: 33–38.
- [8] Yang Y, Adaptive regression by mixing, *Journal of the American Statistical Association*, 2001, **96**(454): 574–588.
- [9] Yuan Z and Yang Y, Combining linear regression models: When and how?, *Journal of the American Statistical Association*, 2005, **100**(472): 1202–1214.
- [10] Bates J M and Granger C W, The combination of forecasts, *Journal of the Operational Research Society*, 1969, **20**(4): 451–468.
- [11] Longford N T, Editorial: Model selection and efficiency — Is ‘which model ...?’ the right question?, *Journal of the Royal Statistical Society Series A: Statistics in Society*, 2005, **168**(3): 469–472.
- [12] You K, Wang X, Long M, et al., Towards accurate model selection in deep unsupervised domain adaptation, *International Conference on Machine Learning (ICML)*, PMLR, 2019, 7124–7133.
- [13] Srivastava N, Hinton G, Krizhevsky A, et al., Dropout: A simple way to prevent neural networks from overfitting, *Journal of Machine Learning Research*, 2014, **15**(1): 1929–1958.
- [14] Zhang X and Zou G, Model averaging method and its application in forecast, *Statistical Research*, 2011, **28**(6): 97–102.
- [15] Leung G and Barron A R, Information theory and mixing least-squares regressions, *IEEE Transactions on Information Theory*, 2006, **52**(8): 3396–3410.
- [16] Li G, Liu Z, Zhang J, et al., Bayesian model averaging by combining deep learning models to improve lake water level prediction, *Science of the Total Environment*, 2024, **906**: 167718.
- [17] Andrew G W and Pavel I, Bayesian deep learning and a probabilistic perspective of generalization, *Advances in Neural Information Processing Systems (NIPS)*, 2020, **33**: 4697–4708.
- [18] Maddox W J, Garipov T, Izmailov P, et al., A simple baseline for bayesian uncertainty in deep learning, *Advances in Neural Information Processing Systems (NIPS)*, 2019, 13153–13164.
- [19] Ganaie M A, Hu M, Malik A K, et al., Ensemble deep learning: A review, *Engineering Applications of Artificial Intelligence*, 2022, **115**: 105151.
- [20] Cheng T C F, Ing C K, and Yu S H, Toward optimal model averaging in regression models with time series errors, *Journal of Econometrics*, 2015, **189**(2): 321–334.
- [21] Hansen B E, Least squares model averaging, *Econometrica*, 2007, **75**(4): 1175–1189.
- [22] Wan A T K, Zhang X, and Zou G, Least squares model averaging by Mallows criterion, *Journal of Econometrics*, 2010, **156**(2): 277–283.
- [23] Liu Q and Okui R, Heteroscedasticity-robust C_p model averaging, *The Econometrics Journal*, 2013, **16**(3): 463–472.
- [24] Zhang H, Liu Z, and Zou G, Least squares model averaging for distributed data, *Journal of*

- Machine Learning Research*, 2023, **24**(215): 1–59.
- [25] Gao Y, Zhang X, Wang S, et al., Frequentist model averaging for threshold models, *Annals of the Institute of Statistical Mathematics*, 2019, **71**: 275–306.
- [26] Zhu R, Wan A T K, Zhang X, et al., A Mallows-type model averaging estimator for the varying-coefficient partially linear model, *Journal of the American Statistical Association*, 2019, **114**(526): 882–892.
- [27] Wang J, He J, Liang H, et al., Optimal model average prediction in orthogonal Kriging models, *Journal of Systems Science & Complexity*, 2024, **37**(3): 1080–1099.
- [28] Qiu Y, Xie T, Yu J, et al., Mallows-type averaging machine learning techniques, *Technical Report*, 2020.
- [29] Xu W, Liu W, Bian J, et al., Instance-wise graph-based framework for multivariate time series forecasting, 2021, arXiv: 2109.06489.
- [30] Kingma D P and Ba J, Adam: A method for stochastic optimization, 2014, arXiv: 1412.6980.
- [31] Paszke A, Gross S, Massa F, et al., Pytorch: An imperative style, high-performance deep learning library, *Advances in Neural Information Processing Systems (NIPS)*, 2019, 8026–8037.
- [32] Liaw R, Liang E, Nishihara R, et al., Tune: A research platform for distributed model selection and training, 2018, arXiv: 1807.05118.
- [33] Lai G, Chang W C, Yang Y, et al., Modeling long- and short-term temporal patterns with deep neural networks, *The 41st International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2018, 95–104.
- [34] Bai S, Kolter J Z, and Koltun V, An empirical evaluation of generic convolutional and recurrent networks for sequence modeling, 2018, arXiv: 1803.01271.
- [35] Lai Z, Zhang D, Li H, et al., LightCTS: A lightweight framework for correlated time series forecasting, *Proceedings of the ACM on Management of Data*, 2023, **1**(2): 1–26.